

Full-Stack Developer Assessment

Project: Task Management Application

Build a full-stack task management application with a REST API backend and a connected frontend.

Required Tech Stack

- Frontend: Next.js (mandatory)
- Database: PostgreSQL (mandatory)
- Backend: Go preferred; you may choose another language based on your expertise

Task 1: Backend API

- Set up a REST API with the following endpoints:
 - POST /tasks — create a task (title, description, status, priority, due date)
 - GET /tasks — list tasks with filtering by status and pagination
 - GET /tasks/:id — fetch a single task
 - PATCH /tasks/:id — update a task
 - DELETE /tasks/:id — delete a task
- Persist data in PostgreSQL
- Add input validation on all write endpoints
- Return proper HTTP status codes and consistent error responses

Task 2: Authentication and Authorization

- Implement signup and login using JWT (or session-based auth)
- Hash passwords before storing them
- Protect all task routes
- Ensure users can only view and modify their own tasks
- Persist auth state on the frontend (a page refresh should keep the user logged in)

Task 3: Frontend

- Task list view with status filter and pagination
- Create and edit task form with client-side validation
- Mark tasks as complete and delete tasks from the UI
- Handle loading, empty, and error states gracefully
- Responsive layout that works on mobile and desktop

Task 4: Search and Sort

- Search tasks by title
- Sort tasks by due date, priority, and created date
- Filters, search, and sort should work together

Task 5: Deliverables

- Working repository with clear setup instructions in the README
- .env.example listing all required environment variables
- At least 3 meaningful tests (backend or frontend)
- Clean, readable commit history

Plus Features (Bonus)

- Role-based access: add an admin role that can view all users' tasks
- Real-time updates: reflect task changes live using WebSockets / SSE
- Optimistic UI: update the UI before the server confirms, with rollback on failure
- Task attachments: allow file uploads on tasks (image or document)
- Activity log: track and display a history of changes per task
- Dockerized setup: provide a docker-compose for one-command local setup
- CI pipeline: set up GitHub Actions to run tests on push
- Dark mode: theme toggle with a persisted preference

Submission Guidelines

- Time expectation: 3–5 days
- Include any assumptions or trade-offs in the README

Email your submission to sabir@rival.io with the following:

- GitHub repository URL (public, or with access granted)
- Deployed live link (frontend + backend)